

Introduction

Two-player game problems show up in coding interviews specifically because they punish the instinct to code first and think later. Predict the Winner looks like a simple array puzzle, but it's really testing whether you can reason about an opponent who is actively working against you — and whether you know how to compress that adversarial reasoning into a state you can actually compute.

This is also one of the most approachable entry points into minimax, the same core idea that powers chess engines and other adversarial search algorithms. You don't need any game-theory background to follow it. You need one array, one recursive function, and one clever substitution that turns "track two players' scores" into "track a single number." That substitution is the entire lesson, and once it clicks, a whole family of two-player DP problems stops looking scary.

Problem Overview

You're given an array of non-negative integers. Two players alternate turns, Player 1 first. On each turn, a player removes either the leftmost or the rightmost number from what remains of the array and adds it to their own score. Both players play optimally — neither ever makes a move that isn't in their own best interest. After all numbers are taken, you compare the two final scores.

The question: can Player 1 guarantee a score that is greater than or equal to Player 2's, assuming Player 2 also plays as well as possible? A tie counts as a win for Player 1 — that detail matters later.

Example

Take `nums = [1, 5, 2]`. Both players start at 0.

If Player 1 greedily grabs the 2 (the bigger visible end), Player 2 is left with `[1, 5]` and takes the 5, leaving Player 1 the 1. Final scores: Player 1 has $2 + 1 = 3$, Player 2 has 5. Player 1 loses.

If instead Player 1 takes the 1 first, Player 2 is left with `[5, 2]`. Player 2 will take the 5 (their best move), leaving Player 1 the 2. Final scores: Player 1 has $1 + 2 = 3$, Player 2 has 5. Still a loss.

Neither first move saves Player 1 here — and that's fine, the answer for this array is genuinely "Player 1 loses." The point of walking through it is to notice that the obvious move (grab the bigger number) didn't help, which is the first hint that we need to look further ahead than one turn.

Intuition

The instinct to simulate every possible sequence of moves is natural, but it explodes fast. On every turn there are two choices — left or right — and with up to 20 numbers in this problem, that's up to 2^{20} possible games, over a million branches to check. We need to shrink the problem, not brute-force it.

The key realization: we don't need to track "my score" and "opponent's score" as two separate numbers. We only need to track their difference — my score minus theirs. Here's why that's valid: every number one player takes is a number the other player didn't get. In a fixed-sum game like this, my gain is mathematically your loss. So the entire outcome of the game can be summarized by one signed number. If that number is zero or positive at the end, Player 1 wins.

This reframing is what makes the problem solvable efficiently. Instead of asking "what will each player's final score be," we ask a narrower, self-contained question: for any remaining slice of the array, what's the best score-difference the player whose turn it is can force?

Brute Force Solution

Idea: Recursively simulate both choices (take left, take right) at every turn, for both players, and check every possible final outcome.

Algorithm: At each state, try both moves, recurse into the resulting sub-array, and pick whichever move gives the current player the better final outcome.

Advantages: Conceptually simple, guaranteed correct, easy to reason about.

Disadvantages: No memoization means the same sub-problems (same remaining slice of the array) get recomputed repeatedly from different branches of the recursion tree. This is what makes it exponential.

Time complexity: $O(2^n)$ — two choices per turn, n turns. **Space complexity:** $O(n)$ for the recursion stack (excluding the exponential blowup in total calls).

Optimal Solution

We define `dp(i, j)` as: given only the sub-array from index `i` to index `j`, what is the best score-difference the player whose turn it is can guarantee?

The player has two choices:

- **Take the left number** (`nums[i]`): their result is `nums[i] - dp(i+1, j)` .
- **Take the right number** (`nums[j]`): their result is `nums[j] - dp(i, j-1)` .

We subtract `dp(i+1, j)` (or `dp(i, j-1)`) because that recursive call returns the *best difference for whoever moves next* — and whoever moves next is now our opponent. Their advantage is our disadvantage, so we subtract it to flip it back into our own frame of reference.

The player picks whichever option is larger:

```
dp(i, j) = max(nums[i] - dp(i+1, j), nums[j] - dp(i, j-1))
```

Base case: when `i == j` , there's only one number left — no decision to make, so `dp(i, j) = nums[i]` .

Because `dp(i, j)` only depends on smaller ranges (`i+1, j` and `i, j-1`), we can safely cache every result the first time it's computed. The same (`i, j`) pair gets asked for repeatedly from different branches of the recursion tree, and reusing the cached answer is what collapses the runtime from exponential to $O(n^2)$.

Player 1 wins if `dp(0, n-1) >= 0` .

Python Code

```
from functools import lru_cache
from typing import List

def predict_winner(nums: List[int]) -> bool:
    @lru_cache(None)
    def dp(i: int, j: int) -> int:
        if i == j:
            return nums[i]

        pick_left = nums[i] - dp(i + 1, j)
        pick_right = nums[j] - dp(i, j - 1)
        return max(pick_left, pick_right)

    return dp(0, len(nums) - 1) >= 0
```

Code Walkthrough

- `@lru_cache(None)` memoizes every `(i, j)` result automatically, so we never recompute a sub-problem we've already solved.
- `if i == j: return nums[i]` is the base case — a single remaining number, no decision.
- `pick_left` is the outcome of taking `nums[i]`, minus whatever the opponent can force on the remaining `nums[i+1:j+1]`.
- `pick_right` mirrors that for taking `nums[j]`.
- `max(pick_left, pick_right)` lets the current player choose whichever move leaves them better off.
- The final `dp(0, len(nums) - 1) >= 0` checks whether Player 1's best possible score-difference over the whole array is zero or positive — including the tie-counts-as-win rule.

Dry Run

Using `nums = [1, 5, 233, 7]`:

Range	Value	How
<code>dp(0,0)</code>	1	base case
<code>dp(1,1)</code>	5	base case
<code>dp(2,2)</code>	233	base case
<code>dp(3,3)</code>	7	base case
<code>dp(0,1)</code>	4	<code>max(1-5, 5-1)</code>
<code>dp(1,2)</code>	228	<code>max(5-233, 233-5)</code>
<code>dp(2,3)</code>	226	<code>max(233-7, 7-233)</code>
<code>dp(0,2)</code>	229	<code>max(1-228, 233-4)</code>
<code>dp(1,3)</code>	-221	<code>max(5-226, 7-228)</code>
<code>dp(0,3)</code>	222	<code>max(1-(-221), 7-229)</code>

`dp(0,3) = 222 ≥ 0`, so Player 1 wins. Sanity check: actual scores end up 234 for Player 1 and 12 for Player 2 — a difference of 222, exactly matching the DP result.

Self-check: try `nums = [2, 2]` yourself before reading on. `dp(0,1) = max(2 - dp(1,1), 2 - dp(0,0)) = max(2-2, 2-2) = 0`. A tie — which still counts as a win for Player 1.

Complexity Analysis

Time: $O(n^2)$ — there are roughly $n^2/2$ distinct `(i, j)` pairs, and each is computed once thanks to memoization.

Space: $O(n^2)$ for the cache, plus $O(n)$ for the recursion stack.

These bounds are correct because memoization guarantees each (i, j) state is evaluated exactly once, and the number of valid states (where $i \leq j$) is bounded by n^2 .

Alternative Solutions

A bottom-up (iterative) version fills a 2D table in order of increasing range width, avoiding recursion entirely — same $O(n^2)$ time and space, but no call-stack overhead. A further optimization compresses the 2D table into a 1D rolling array, since each row of the DP table only depends on the previous row, bringing space down to $O(n)$. Given this problem's constraint of at most 20 numbers, none of these optimizations are necessary in practice — they're worth mentioning if an interviewer asks for a follow-up.

Edge Cases

- **Single element:** Player 1 takes it and automatically wins — $dp(i, i) = \text{nums}[i]$, which is always ≥ 0 for non-negative inputs.
- **All zeros:** every difference resolves to 0, which counts as a win for Player 1 under the tie rule.
- **Two equal elements:** as shown above, it resolves to a 0 difference — a tie, still a win.
- **Maximum size (20 elements):** even at $O(n^2)$, this finishes essentially instantly.
- **Large value disparities:** the subtraction can produce large negative or positive intermediate values; Python's arbitrary-precision integers handle this without overflow concerns.

Common Mistakes

1. **Using a greedy strategy** — always grabbing the bigger visible end. This ignores that taking a number can expose an even bigger one for the opponent next turn.
2. **Tracking two separate scores** instead of one difference, which makes the recursion far messier and easier to get wrong.
3. **Adding instead of subtracting** the opponent's result — forgetting that their best outcome is your worst outcome.
4. **Forgetting the tie-counts-as-a-win rule**, and checking for $dp(0, n-1) > 0$ instead of ≥ 0 .
5. **Skipping memoization**, leading to exponential blowup even though the recurrence itself is correct.
6. **Off-by-one errors** in the base case, such as checking $i > j$ instead of $i == j$.

Interview Questions

- How would you convert this top-down solution into a bottom-up, iterative DP table?
- Can you reduce the space complexity from $O(n^2)$ to $O(n)$?
- How does this generalize if more than two players are involved?
- What changes if players could take from three positions instead of two ends?
- Why is the "difference trick" valid — under what conditions does it break down?

Similar Problems

- **Stone Game (LeetCode 877)** — nearly identical mechanics, restricted to even-length arrays with a guaranteed win.
- **Stone Game II / III** — same family, with variable take-sizes per turn.
- **Optimal Strategy for a Game** — a classic formulation of this exact problem outside LeetCode.
- **Burst Balloons** — different mechanics, but the same interval DP pattern (`dp(i, j)` over sub-ranges).
- **Matrix Chain Multiplication** — another interval DP problem that builds bigger ranges from smaller, already-solved ones.

Key Takeaways

The greedy instinct fails because it can't see beyond the current move. Reframing the problem around a single score-difference instead of two separate scores turns an adversarial two-player game into a self-contained recursive question. Memoizing `dp(i, j)` is what makes that recursion computationally feasible, and the subtraction in the recurrence is simply "my opponent's advantage is my disadvantage" written as math.

Watch the Video

For the full walkthrough with the cell-by-cell dry run and a self-check example, watch the video here: {LINK}

About the Series

This is part of Fun with Learning Technology, a series that works through one interview problem at a time — building intuition first, writing the code second, and never skipping the "why" in between.

Call To Action

If this helped the DP click for you, a like on the video helps this channel grow, and subscribing means you won't miss the next one in the series. Subscribe to the Substack for the written breakdowns alongside the videos, and drop a comment with the problem you'd like covered next — I read all of them.

Quick Review

You choose 2 numbers from either end of a row optimally — what pattern applies?

Minimax with game theory: maximize your gain while opponent minimizes it, i.e. minimizes your future gain.

Time complexity of the DP solution for Predict the Winner?

$O(n^2)$ — filling a 2D table over subarray start/end pairs.

Space complexity of the optimized DP for this problem?

$O(n)$ if you use a 1D rolling array instead of a full 2D table.

Why can't you just simulate every possible sequence of picks?

Branching factor 2 per turn gives $O(2^n)$ sequences — exponential blowup for larger arrays.

What's the 'track one number' trick instead of two scores?

Store the score DIFFERENCE (current player minus opponent) per subarray; positive diff at the end means player one wins.

Follow-up: how would you return the actual winning move sequence, not just win/lose?

Backtrack through the DP table from the top, at each step choosing whichever end matches the recorded optimal choice.